



TEORÍA DE ALGORITMOS
(75.29) CURSO BUCHWALD - GENENDER

Trabajo Práctico 3

Problemas NP-Completos

11 de junio de 2026

Alejandro, Pablo Martin
98021

Prada, Matias Leonel
99397

Poli Salvador, Ignacio Manuel
110399

1. Consideraciones

Este documento está pensado para que sirva de guía, e incluso template a la hora de realizar un informe para la materia (por no decir, cualquier documento formal en la facultad). La idea es utilizar $\text{\LaTeX}$, pura y exclusivamente para que tenga un formato acorde. Es decir, no hecho en un simple procesador de texto (como pudiese ser Word).

Aquí sólo se mostraran algunas de las cosas que se pueden hacer con $\text{\LaTeX}$. Si se quiere hacer algo específico, o de otra forma, abunda la documentación (y artículos de StackOverflow) al respecto.

Definimos ciertos lineamientos técnicos a considerar para los informes:

- En el informe debe estar incluido el código más importante (es decir, los algoritmos, no un main) cuando corresponda.
- Debe estar explicado cómo se hacen los sets de prueba. Inclusive, si notan alguna falencia con esta generación que pueda afectar de alguna forma el análisis del resultado, se espera que lo enuncien (hay trabajos donde esto no sucederá, pero hay trabajos donde sí, especialmente cuando vemos aproximaciones).
- Los gráficos deben estar bien generados: deben tener título, los ejes deben tener tanto qué refieren como qué unidad de medida usan (en caso de tenerla). En caso de tener gráficos superpuestos, debe quedar claro qué es cada gráfico.
- **Importante:** en caso de tener que realizar una reentrega (y por ende, tener que cambiar cosas del informe), salvo que haya que rehacer una enorme cantidad del mismo, no modificar el informe anterior sino agregar un anexo al final indicando las correcciones realizadas en la nueva entrega, para agilizar las correcciones.

A su vez, dejamos algunas breves líneas sobre cuestiones de redacción:

- Evitar errores ortográficos y gramaticales.
- El lenguaje debe ser técnico. No tiene que por eso ser un texto aburrido, pero evitar frases como *"la cosa que más me complicó fue..."*, *"creo que..."*.
- Usar plural de modestia, incluso si el trabajo fuera realizado por una única persona.

2. Análisis del Problema

El presente trabajo plantea el desafío de seleccionar un conjunto de jugadores para disputar un partido amistoso con el objetivo estratégico de apaciguar las presiones de la prensa. Para lograrlo, el equipo técnico debe asegurarse de contentar a diferentes medios periodísticos, seleccionando al menos un jugador del agrado de cada uno de ellos, pero comprometiendo la menor cantidad de cupos posibles en el equipo.

Formalizando el escenario, podemos definir los elementos del problema de la siguiente manera:

- Un conjunto universal A conformado por los n jugadores convocados (en este caso, los 43 jugadores de la lista).
- Una serie de m subconjuntos $B_1, B_2, \dots, B_m$, donde cada $B_i \subseteq A$ representa el subconjunto de jugadores que exige ver en cancha el i -ésimo medio de prensa.

El objetivo original del problema consiste en encontrar un conjunto $C \subseteq A$ de tamaño mínimo tal que logre intersectar a todos los subconjuntos de preferencias periodísticas. Es decir, se debe garantizar la siguiente condición:

$$C \cap B_i \neq \emptyset \quad \forall i \in \{1, 2, \dots, m\} \quad (1)$$

Este escenario modela un problema clásico de la teoría de la complejidad computacional y optimización combinatoria conocido como el **Hitting-Set Problem**.

Sin embargo, para los fines de las demostraciones teóricas iniciales de este trabajo práctico (demostración de pertenencia a NP y prueba de NP-Complejidad), es necesario abstraerse de la minimización del conjunto. Por lo tanto, abordaremos y utilizaremos estrictamente la **versión de decisión** del Hitting-Set Problem.

En esta versión, se introduce una cota superior k (que en nuestro dominio representa la cantidad máxima de jugadores que el técnico está dispuesto a "gastar" para contentar a la prensa) y el problema se reduce a responder afirmativa o negativamente a la siguiente pregunta:

Dado el conjunto A de n elementos, los m subconjuntos $B_1, B_2, \dots, B_m$ de A , y un número entero k , ¿existe un subconjunto $C \subseteq A$ con $|C| \leq k$ tal que C tenga al menos un elemento de cada B_i ?

2.1. Algoritmo por División y Conquista

A continuación, mostramos la implementación de un algoritmo que encuentra el máximo de un arreglo por División y Conquista. Es decir, busca el máximo que corresponde al subarreglo izquierdo, lo mismo para el derecho y se queda con el máximo entre ambos *sub máximos*.

```
1 def maximo(datos):
2     if len(datos) == 1:
3         return 0
4
5     izq = maximo(datos[:len(datos)//2])
6     der = maximo(datos[len(datos)//2:])
7     return izq if izq > der else der
```

La ecuación de recurrencia que corresponde a este algoritmo es:

$$\mathcal{T}(n) = 2\mathcal{T}\left(\frac{n}{2}\right) + \mathcal{O}(n)$$

Esto es porque tenemos 2 llamados recursivos, cada lado con la mitad del problema, y al partir hacemos una slice, lo cual en Python realiza una copia, por lo cual demora tiempo lineal en aplicarse en cada caso.

Aplicando el teorema maestro, la complejidad nos queda en $\mathcal{O}(n \log n)$. En este caso, nos quedó peor complejidad que en el caso iterativo pura y exclusivamente por abusar del lenguaje de programación sin tomar en cuenta el tiempo que consume hacer un slice. Dejando de hacer esto, podemos mostrar la siguiente versión del algoritmo:

3. Conclusiones

Acá irían las conclusiones de todo nuestro trabajo :)